

Syntax:

- It refers to the rules and regulations for writing any statement in a programming language like C/C++.
- It does not have to do anything with the meaning of the statement.
- A statement is syntactically valid if it follows all the rules.
- It is related to the grammar and structure of the language.

Semantics:

- It refers to the meaning associated with the statement in a programming language.
- It is all about the meaning of the statement which interprets the program easily.
- Errors are handled at runtime.

Syntax, Semantics, and Pragmatics

- In the linguistics of both natural and computer languages, the terms *syntax*, *semantics* and *pragmatics* are used to categorize descriptions of language characteristics.
- The *syntax* of a language describes the structure and composition of allowable phrases and sentences of the language.
- But syntax itself is devoid of meaning, simply telling us what strings are valid and how they may be parsed or decomposed.
- The meaning of these syntactic elements must be provided through *semantics*.
- In essence, we may think of providing syntactic elements as inputs to a *semantic function*, which in turn provides some representation of the meaning of the elements as output.
- **Pragmatics** is the third general area of language description, referring to practical aspects of how constructs and features of a language may be used to achieve various objectives.
- Consider, for example, the syntax, semantics and pragmatics of an assignment statement.
- As a syntactic construct, an assignment statement may consist of a variable and an expression (themselves syntactic constructs), separated by the token = as an assignment operator.
- Semantically, the variable denotes a location in computer memory, while the expression denotes computation of a value based on the contents of memory.
- Overall, the semantics of assignment is to perform the expression evaluation based on current memory contents and then update the value stored in the particular location corresponding to the variable.

1.2 Syntax, Semantics, and Pragmatics

Programming languages are traditionally viewed in terms of three facets:

1. **Syntax** — the *form* of programming languages.
2. **Semantics** — the *meaning* of programming languages.
3. **Pragmatics** — the *implementation* of programming languages.

Here we briefly describe these facets.

Syntax

Syntax focuses on the concrete notations used to encode programming language phrases. Consider a phrase that indicates the sum of the product of v and w and the quotient of y and z . Such a phrase can be written in many different notations — as a traditional mathematical expression:

$$vw + y/z$$

or as a LISP parenthesized prefix expression:

$$(+ (* v w) (/ y z))$$

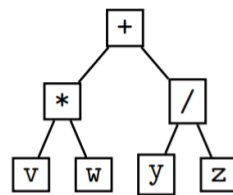
or as a sequence of keystrokes on a postfix calculator:

V	ENTER	W	ENTER	×	Y	ENTER	Z	ENTER	÷	+
---	-------	---	-------	---	---	-------	---	-------	---	---

or as a layout of cells and formulas in a spreadsheet:

	1	2	3	4
A	v=		v*w =	A2 * B2
B	w=		y/z =	C2 / D2
C	y=		ans =	A4 + B4
D	z=			

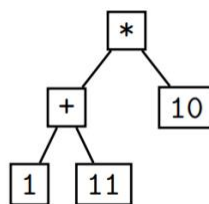
or as a graphical tree:



Although these concrete notations are superficially different, they all designate the same abstract phrase structure (the sum of a product and a quotient). The syntax of a programming language specifies which concrete notations (strings of characters, lines on a page) in the language are legal and which tree-shaped abstract phrase structure is denoted by each legal notation.

Semantics

Semantics specifies the mapping between the structure of a programming language phrase and what the phrase means. Such phrases have no inherent meaning: their meaning is determined only in the context of a system for interpreting their structure. For example, consider the following expression tree:



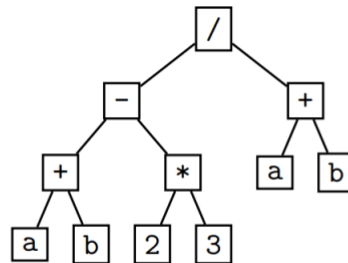
Suppose we interpret the nodes labeled 1, 10, and 11 as the usual decimal notation for numbers, and the nodes labeled + and * as the sum and product of the values of their subnodes. Then the root of the tree stands for $(1 + 11) \cdot 10 = 120$. But there are many other possible meanings for this tree. If * stands for exponentiation rather than multiplication, the meaning of the tree could be 12^{10} . If the numerals are in binary notation rather than decimal notation, the tree could stand for (in decimal notation) $(1 + 3) \cdot 2 = 8$. Alternatively, suppose that odd integers stand for the truth value *true*, even integers stand for the truth value *false*, and + and * stand for, respectively, the logical disjunction ($\vee$) and conjunction ($\wedge$) operators on truth values; then the meaning of the tree is *false*. Perhaps the tree does not indicate an evaluation at all, and only stands for a property intrinsic to the tree, such as its height (3), its number of nodes (5), or its shape (perhaps it describes a simple corporate hierarchy). Or maybe the tree is an arbitrary encoding for a particular object of interest, such as a person or a book.

This example illustrates how a single program phrase can have many possible meanings. Semantics describes the relationship between the abstract structure of a phrase and its meaning.

Pragmatics

Whereas semantics deals with *what* a phrase means, pragmatics focuses on the details of *how* that meaning is computed. Of particular interest is the effective use of various resources, such as time, space, and access to shared physical devices (storage devices, network connections, video monitors, printers, speakers, etc.).

As a simple example of pragmatics, consider the evaluation of the following expression tree (under the first semantic interpretation described above):



Suppose that **a** and **b** stand for particular numeric values. Because the phrase $(+ \ a \ b)$ appears twice, a naive evaluation strategy will compute the same sum twice. An alternative strategy is to compute the sum once, save the result, and use the saved result the next time the phrase is encountered. The alternative strategy does not change the meaning of the program, but does change its use of resources; it reduces the number of additions performed, but may require extra storage for the saved result. Is the alternative strategy better? The answer depends on the details of the evaluation model and the relative importance of time and space.

What are Variables In Programming?

*A Variables In Programming is a **named storage location** that holds a value*

Basis	Syntax	Semantics
Meaning	It refers to the rules of any statement in the programming language.	It refers to the meaning associated with any statement in the programming language
Error	It is referred to as a syntax error. It is generally encountered at the compile time. It occurs when a statement that is not valid according to the grammar of the programming language. Some examples are missing semicolons in C++, using undeclared <u>variables</u> in Java, etc.	It referred to as a semantic error. It is generally encountered at run time. It occurs when a statement is syntactically valid but does not do what the programmer intended. This type of error is tough to catch.
In linguistics	The syntax is the arrangement or order of words, determined by both the writer's style and grammar rules.	There are two areas of semantics that are logical semantics and lexical semantics.
Sensitivity	the syntax is sensitive in most programming languages.	Most of the semantics are case-insensitive.

or data. These values can change during the execution of a program, hence

What are Variables In Programming?

A Variables In Programming is a **named storage location** that holds a value the term “variable.” Variables are essential for storing and manipulating data in computer programs. A variable is the basic building block of a program that can be used in expressions as a substitute in place of the value it stores.

Declaration of Variables In Programming:

In programming, the declaration of variables involves specifying the **type and name** of a variable before it is used in the program. The syntax can vary slightly between programming languages, but the fundamental concept remains consistent.

- `#include <iostream>`
- `using namespace std;`
-
- `int main()`
- `{`
- `// Syntax: datatype variable_name;`
- `int age;`
- `double price;`
- `char grade;`
-
- `return 0;`
- `}`

Initialization of Variables In Programming:

Initialization of variables In Programming involves **assigning an initial value** to a declared variable. The syntax for variable initialization varies across programming languages.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    // Declaration and Initialization
```

```
    int age = 25;
```

```
    double price = 10.99;
```

```
    char grade = 'A';
```

```
    return 0;
```

```
}
```

1. Global Variables:

Global variables in programming are declared **outside any function or block** in a program and **accessible throughout the entire codebase**. In simpler words,

Global variables can be accessed in any part of the program, including functions, blocks, or modules.

2. Local Variables:

Local variables in programming are **declared within a specific function, block, or scope** and are only **accessible within that limited context**. In simpler words, Local variables are confined to the block or function where they are declared and cannot be directly accessed outside that scope.

Difference between Local variables and Global Variables:

Characteristic	Global Variables	Local Variables
Scope1	Accessible throughout the entire codebase	Confined to the block, function, or scope of declaration.
Visibility	Accessible by any part of the program, including functions, blocks, or modules	Accessible only within the limited context of declaration.
Lifetime	Exist for the entire duration of the program	It exists only during the execution of the block or function.
Initialization	May have a default value, can be initialized outside functions or blocks	May not have a default value, and must be explicitly initialized within the scope.
Access from Functions	Accessible directly from any function or block	Directly accessible only within the declaring function

Naming Conventions:

Naming conventions for variables In Programming help maintain code readability and consistency across programming projects. While specific conventions can vary between programming languages, some common practices are widely followed. Here are general guidelines for naming variables:

- **Descriptive and Meaningful:** Choose names that indicate the purpose or content of the variable. A descriptive name improves code readability and understanding.

- **Camel Case or Underscore Separation:** Use camel case (myVariableName) or underscores (my_variable_name) to separate words in a variable name. Choose one style and stick to it consistently.
- **Avoid Single Letter Names:** Except for loop counters or well-known conventions (e.g., i for an index), avoid single-letter variable names. Use names that convey the variable's purpose.
- **Follow Language Conventions:** Adhere to the naming conventions recommended by the programming language you are using. For example, Java typically uses camel case (myVariableName), while Python often uses underscores (my_variable_name).
- **Avoid Reserved Words:** Avoid using reserved words or keywords of the programming language as variable names.
- **Avoid Abbreviations:** Use full words instead of abbreviations, except when widely accepted or for standard terms (e.g., max, min).

Scope of a variable:

The scope of a variable in programming refers to the region of the program where the variable can be accessed or modified. It defines the visibility and lifetime of a variable within a program. There are typically two types of variable scope:

Local Scope:

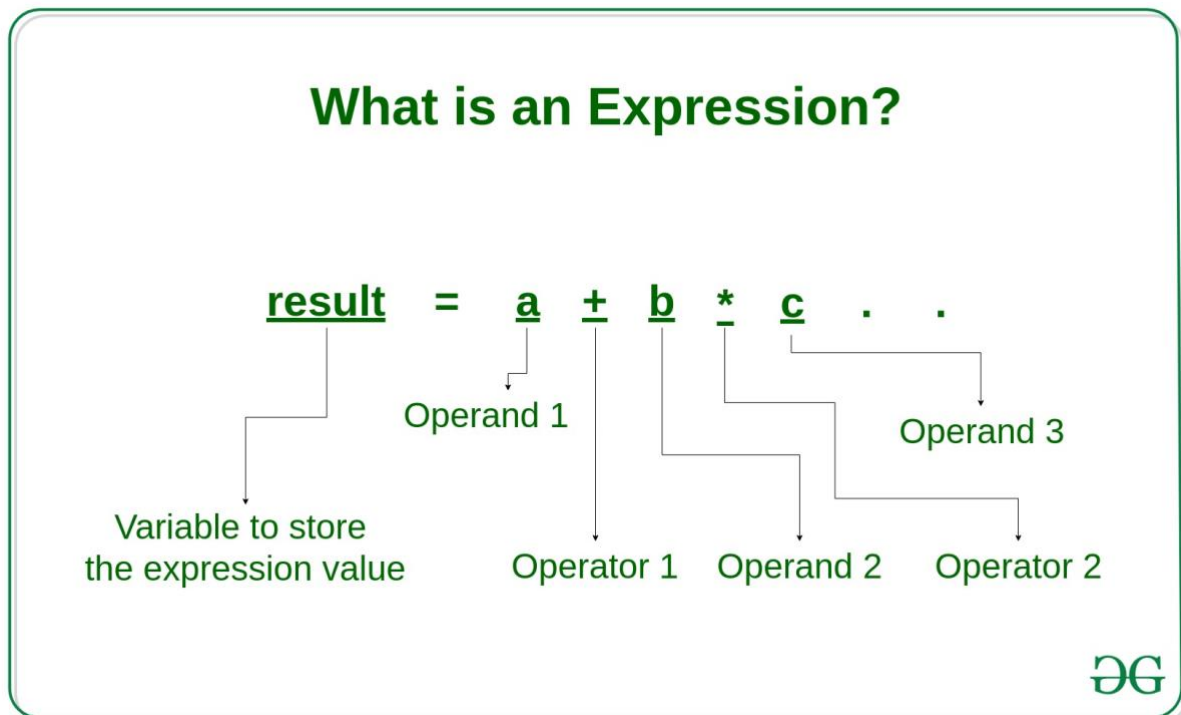
- A variable declared inside a function, block, or loop has a local scope.
- It can only be accessed within the specific function, block, or loop where it is declared.
- Its lifetime is limited to the duration of the function, block, or loop.

Global Scope:

- A variable declared outside of any function or block has a global scope.
- It can be accessed from anywhere in the program, including inside functions.
- Its lifetime extends throughout the entire program.
- Understanding and managing variable scope is crucial for writing maintainable and bug-free code, as it helps avoid naming conflicts and unintended side effects.
- In conclusion, variables in programming are fundamental elements in programming, serving as named storage locations to hold and manipulate data. They are crucial in writing dynamic, adaptable, and functional code. The proper use of variables, adherence to clear naming conventions, and understanding of scope contribute to code readability, maintainability, and scalability.

Expression:

An expression is a combination of operators, constants and variables. An expression may consist of one or more operands, and zero or more operators to produce a value.

**Example:**

`a+b`

`c`

`s-1/7*f`

`.`

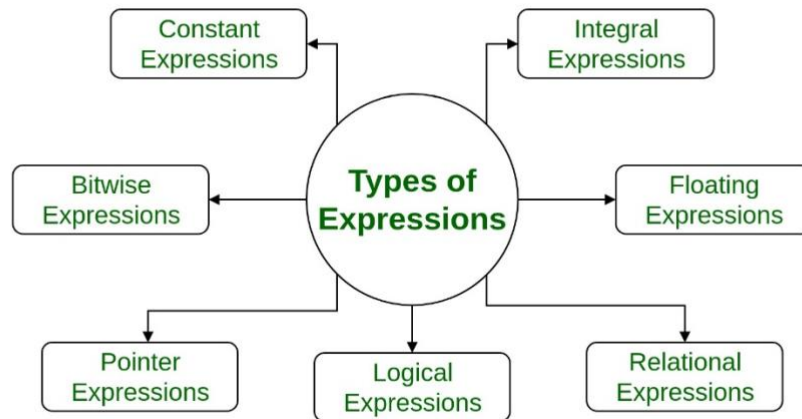
`.`

etc

Types of Expressions:

Expressions may be of the following types:

Types of Expressions



- **Constant expressions:** Constant Expressions consists of only constant values. A constant value is one that doesn't change.
Examples:
5, $10 + 5 / 6.0$, 'x'
- **Integral expressions:** Integral Expressions are those which produce integer results after implementing all the automatic and explicit type conversions.
Examples:
 x , $x * y$, $x + \text{int}(5.0)$
where x and y are integer variables.
- **Floating expressions:** Float Expressions are which produce floating point results after implementing all the automatic and explicit type conversions.
Examples:
 $x + y$, 10.75
where x and y are floating point variables.
- **Relational expressions:** Relational Expressions yield results of type bool which takes a value true or false. When arithmetic expressions are used on either side of a relational operator, they will be evaluated first and then the results compared. Relational expressions are also known as Boolean expressions.
Examples:
 $x \leq y$, $x + y > 2$

- **Logical expressions:** Logical Expressions combine two or more relational expressions and produces bool type results.
Examples:
`x > y && x == 10, x == 10 || y == 5`
- **Pointer expressions:** Pointer Expressions produce address values.
Examples:
`&x, ptr, ptr++`
where x is a variable and ptr is a pointer.
- **Bitwise expressions:** Bitwise Expressions are used to manipulate data at bit level. They are basically used for testing or shifting bits.
Examples:
`x << 3`
shifts three bit position to left
`y >> 1`
shifts one bit position to right.
Shift operators are often used for multiplication and division by powers of two.

Note: An expression may also use combinations of the above expressions. Such expressions are known as **compound expressions**.

A compiler is a program that converts HLL(High-Level Language) to LLL(Low-Level Language) like machine language. In a compiler, there is a need for storage allocation strategies in Compiler design because it is very important to use the right strategy for storage allocation as it can directly affect the performance of the software.

Storage Allocation Strategies

There are mainly three types of Storage Allocation Strategies:

1. Static Allocation
2. Heap Allocation
3. Stack Allocation

1. Static Allocation

Static allocation lays out or assigns the storage for all the data objects at the compile time. In static allocation names are bound to storage. The address of these identifiers will be the same throughout. The memory will be allocated in a static location once it is created at compile time. C and C++ use static allocation.

For example:

```
int number = 1;
static int digit = 1;
```

Advantages of Static Allocation

1. It is easy to understand.
2. The memory is allocated once only at compile time and remains the same throughout the program completion.
3. Memory allocation is done before the program starts taking memory only on compile time.

Disadvantages of Static Allocation

1. Not highly scalable.
2. Static storage allocation is not very efficient.
3. The size of the data must be known at the compile time.

2. Heap Allocation

- Heap allocation is used where the Stack allocation lacks if we want to retain the values of the local variable after the activation record ends, which we cannot do in stack allocation, here LIFO scheme does not work for the allocation and de-allocation of the activation record.
- Heap is the most flexible storage allocation strategy we can dynamically allocate and de-allocate local variables whenever the user wants according to the user needs at run-time. The variables in heap allocation can be changed according to the user's requirement. C, C++, Python, and Java all of these support Heap Allocation.

For example:

```
int* ans = new int[5];
```

Advantages of Heap Allocation

1. Heap allocation is useful when we have data whose size is not fixed and can change during the run time.
2. We can retain the values of variables even if the activation records end.
3. Heap allocation is the most flexible allocation scheme.

Disadvantages of Heap Allocation

1. Heap allocation is slower as compared to stack allocation.
2. There is a chance of memory leaks.
3. Stack Allocation

Stack is commonly known as Dynamic allocation. Dynamic allocation means the allocation of memory at run-time. Stack is a data structure that follows the LIFO principle so whenever there is multiple activation record created it will be pushed or popped in the stack as activations begin and ends. Local variables are bound to new storage each time whenever the activation record begins because the storage is allocated at runtime every time a procedure or function call is made. When the activation record gets popped out, the local variable values get erased because the storage allocated for the activation record is removed. C and C++ both have support for Stack allocation.

For example:

```
void sum(int a, int b){int ans = a+b;cout<<ans;}  
// when we call the sum function in the example above,  
memory will be allotted for the variable ans
```

Conclusion

In conclusion, different storage allocation strategies play an important role in determining the best-fit storage allocation strategy according to the need of the user as it helps in determining how the memory is going to be allocated and deallocated. Different storage allocation strategies have their own advantages and disadvantages and the choice depends on the factors like speed, memory allocation, efficiency, etc. So we can choose the allocation strategy according to the requirement.

In computer programming, a **statement** is a single line of code that performs a specific task. For example, the following line of programming code from the Perl programming language is an example of a statement.

```
$a = 3;
```

In this example statement, a variable (\$a) is assigned the value of "3" that is stored as a string. This type of statement is known as an **assignment statement** because a value is being assigned to a variable.

Lvalue and Rvalue

Overview

Some programming languages use the idea of **l-values** and **r-values**, deriving from the typical mode of evaluation on the left and right hand side of an assignment statement. An lvalue refers to an object that persists beyond a single expression. An rvalue is a temporary value that does not persist beyond the expression that uses it.

Discussion

Lvalue and Rvalue refer to the left and right side of the assignment operator. The **Lvalue** (pronounced: L value) concept refers to the requirement that the operand on the left side of the assignment operator is modifiable, usually a variable. **Rvalue** concept pulls or fetches the value of the expression or operand on the right side of the assignment operator. Some examples:

```
age = 39
```

The value 39 is pulled or fetched (Rvalue) and stored into the variable named age (Lvalue); destroying the value previously stored in that variable.

```
voting_age = 18
```

```
age = voting_age
```

If the expression has a variable or named constant on the right side of the assignment operator, it would pull or fetch the value stored in the variable or constant. The value 18 is pulled or fetched from the variable named voting_age and stored into the variable named age.

```
age < 17
```

If the expression is a test expression or Boolean expression, the concept is still an Rvalue one. The value in the identifier named age is pulled or fetched and used in the relational comparison of less than.

```
JACK_BENNY'S_AGE = 39
```

```
JACK_BENNY'S_AGE = 65;
```

This is illegal because the identifier JACK_BENNY'S_AGE does not have Lvalue properties. It is not a modifiable data object, because it is a constant.

Some uses of the Lvalue and Rvalue can be confusing in languages that support increment and decrement operators. Consider:

```
oldest = 55
```

```
age = oldest++
```

Postfix increment says to use my existing value then when you are done with the other operators; increment me. Thus, the first use of the oldest variable is an Rvalue context where the existing value of 55 is pulled or fetched and then assigned to the variable age; an Lvalue context. The second use of the oldest

variable is an Lvalue context wherein the value of the oldest is incremented from 55 to 56.

Key Terms

Lvalue

The requirement that the operand on the left side of the assignment operator is modifiable, usually a variable.

Rvalue

Pulls or fetches the value stored in a variable or constant.

Constants in Programming Language

A constant is a quantity that does not change its value over a period of time. A constant is a universal quantity, where it remains the same for every aspect of computation. This quantity can be stored at specified locations in the memory of the computer. Once a constant is defined in the program, it cannot be changed. Let us dive deeper into the article and understand the concept of Constants in programming languages in a better way.

Definition of Constants

Constant is an entity that refers to a fixed value of data and which cannot be modified. During execution/computation of programming, the value of a constant cannot be altered, it remains constant.

The most common example which can be considered to understand the concept of a constant is “PI”

PI = 3.1415927 whose value is universal and does not change.

Methods of Defining a Constant in Program

1. The keyword used to define a constant in a program is – **const**

For example –

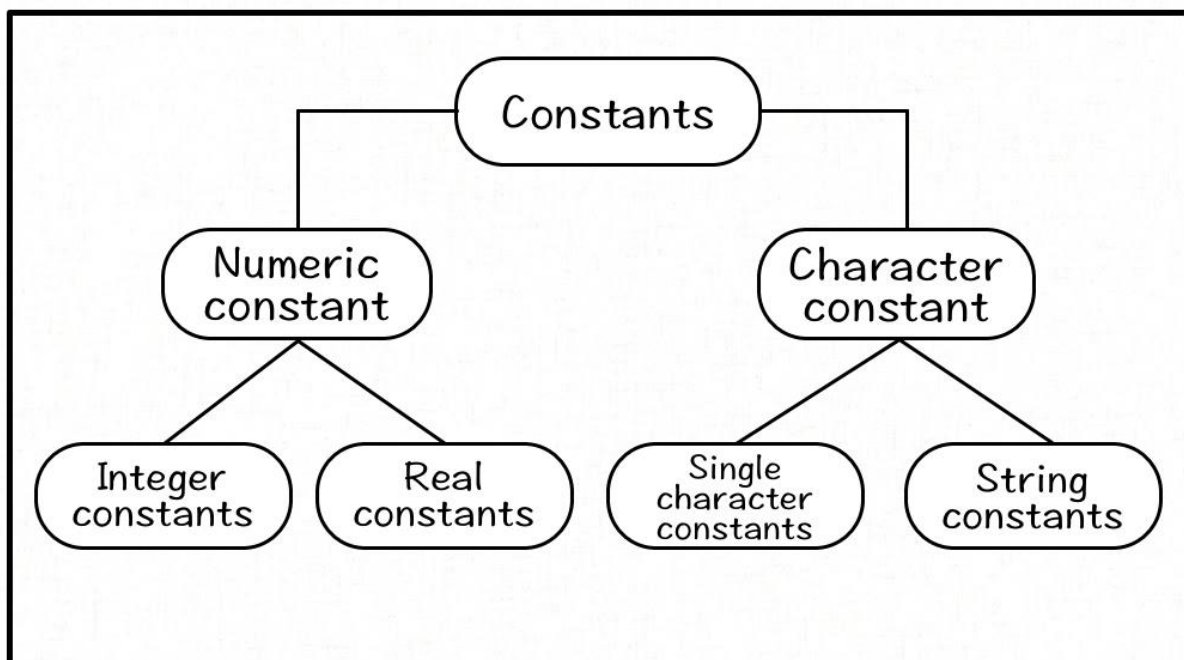
- `const int i = 10; // i is a constant with value 10 which cannot be modified throughout the execution life cycle of a program.`
- `const float PI = 3.1415927; //PI value defined as a constant`

2. Another method of defining a constant is by using the #define preprocessor directive.

Syntax – #define identifiername value //where identifiername = name given to constant

For example –

- #define val 10
- #define char 'G'



Types of Constants

Integer Constants

Integer constants are sequences of integers with a fixed value. They must not contain any decimal points or fractional numbers. Integer constants can either be positive or negative numbers. They include decimal system integers, octal system integers, hexadecimal system integers.

A. **Decimal integers** – Set of digits ranging from 0 to 9. For Example –
const int y = 123;

B. **Octal integers** – Set of digits ranging from 0 to 7 which start with the number '0' in the beginning/at the prefix. For example – const int x = 032;

- C. **Hexadecimal integers** – Set of digits ranging from 0 to 9 and alphabets from A to F where A indicates number 10 and F indicates number 15. 0x is used at the prefix of the value. For example – `const int z = 0x14;`

Browse more Topics Under Data Types, Variables and Constants

- [Concept of Data types](#)
- [Built-in Data Types](#)
- [Access Modifier](#)
- [Variables of Built-in-Datatypes](#)
- [Declaration/Initialization of Variables](#)
- [Assignment Statement](#)
- [Type Modifier](#)

Real Constants or Floating-point Constants

They contain fractional numbers that can be written in 2 forms, Fractional form, and Exponential form.

Eg. 140.9, 4578.218, 4.1 e 45 (i.e. $4.1 * 10^{45}$), 5.0 e -2 (i.e. $5.0 * 10^{-2}$)

Character Constants

Any single character from the defined character set is called a character constant. They include a single alphabet, single-digit, single symbol enclosed within single inverted commas.

Eg. 'A', 'p', '4', '\$', '='

String Constants

It contains a set/collection of characters that should be enclosed within double inverted commas.

Eg. "Hello", "Length", "ND123"

Advantages of Constants

1. **Immutability** – It is useful if you know that the value of the variable will not change throughout your program lifecycle.
2. **Code Maintenance** – If you need to use the Constant value multiple times in your program, assigning it to a variable and then using that variable will save time.
3. **Memory Allocation** – It is generally stored in program memory (ROM) and not in the main memory(RAM) of the computer thus providing faster execution speed.

Concept of Multi-Paradigm programming?

The concept of multi-paradigm programming is very simple to understand. When a programming language use more than one programming paradigm it know as multi-paradigm language, and the concept of combining these paradigms are Multiparadigm programming.

In OOPs, programmers can think of a program as a collection of interacting objects, while in functional programming a program can be thought of as a sequence of stateless function evaluations. When programming computers or systems with many processors, process-oriented programming allows programmers to think about applications as sets of concurrent processes acting upon logically shared data structures

Some languages are designed to support one particular paradigm (Smalltalk supports object-oriented programming, Haskell supports functional programming), while other programming languages support multiple paradigms

Many programming paradigms are as well known for what techniques they forbid as for what they enable. For instance, pure functional programming disallows the use of Side effects while Structured programming disallows the use of the goto statement. Partly for this reason, new paradigms are often regarded as doctrinaire or overly rigid by those accustomed to earlier styles.